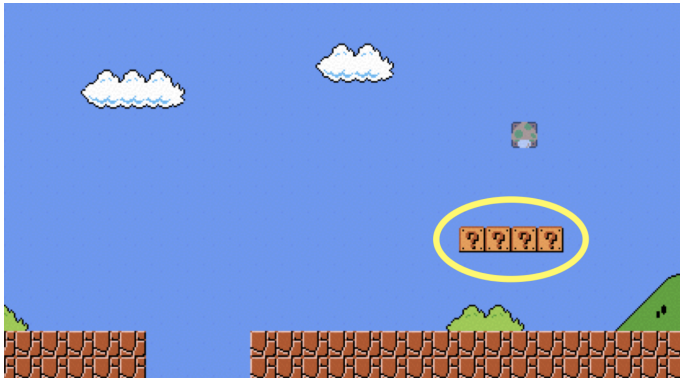
Introdução à Computação
tipos, operadores e precisão

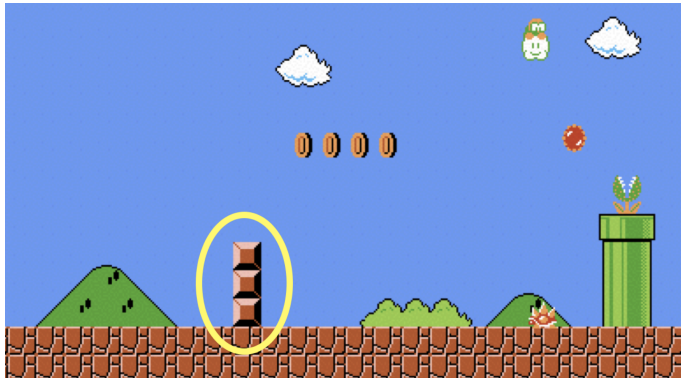
Alexandre Rademaker

Mario I

Vamos imprimir alguns blocos na tela, uma versão simplifica de

https://en.wikipedia.org/wiki/Super_Mario_Bros.





Mario II

src/mario0.c

```c
#include <stdio.h>

int main(void) {

    printf("????\n");
}
```

Veja `src/mario1.c`. Vamos testar? Vamos pedir entrada para usuário?

Mario III

src/mario2.c

```
1  #include <stdio.h>
2
3  int main(void) {
4
5    int n;
6    do {
7        printf("width: "); scanf("%d", &n);
8    }
9    while (n < 1);
10
11   for (int i = 0; i < n; i++) {
12     printf("?");
13   }
14   printf("\n");
15 }
```

Testar? O que ocorre com entradas como: 0, -1, 100, a, a10, 10a etc.

Mario IV

A função `scanf` tenta ler da entrada "buffer entrada" (STDIN) um padrão de caracteres que possa ter interpretado como inteiro. Se conseguir, retorna 1, do contrário retorna 0.

Caracteres não consumidos permanecem no buffer de entrada. Se nenhum caracter for consumido então o valor na memória que deveria ser alimentada como um inteiro pode conter "lixo".

Como saber mais? Como pedir ajuda? aqui e aqui.

Vamos tratar deste erro mais tarde!

Mario V

Vamos imprimir blocos bi-dimensionais?

src/mario3.c

```c
#include <stdio.h>

int main(void)
{
  // leitura
  int n;
  do {
    printf("size: "); scanf("%d", &n);
  } while (n < 1);

  // producao saida
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++)
      printf("#");
    printf("\n");
  }
}
```

Comentários

> *A handy rule of thumb is: a comment should only be added to answer a question that the code can't.*

O excesso de comentários polui o código! Comentários óbvios não tornam programas ruins melhores.

tipos, operadores e precisão I

Tipos referem-se aos possíveis valores que podem ser armazenados em variáveis. Uma variável do tipo `char` pode armazenar caracteres como `'a'` ou mesmo `'2'`.

Os tipos são importante por determinarem o limite de memória usado para cada tipo. O maior valor para `int` é de 2^{32}.

tipos, operadores e precisão II

Vamos lembrar de nossa calculadora.

```
> ./calculator1
x: 2000000000
y: 2000000000
-294967296
```

O tipo 'int' tem um armazenamento limitado, normalmente 32 bits, logo pode apenas conter $2^{32} = 4,294,967,296$ diferentes valores. Mas como valores podem ser positivos ou negativos, nosso limite é $2^{31} = 2,147,483,648$. integer overflow

Vamos mudar para `long` e testar? Vide `src/calculator2.c`

tipos, operadores e precisão III

Veja `src/calculator3.c`. Dividir dois números inteiros não produz um `float` mas um inteiro (truncado).

Isto é o que chamamos de floating-point imprecision, não podemos representar **todos** os números reais com um número finito de bits.

Qualquer representação de ponto-flutuante terá algum limite de precisão.

```
> make calculator3
> ./calculator3
x: 2
y: 3
0.66666668653488159179687500000000000000000000000000
> ./calculator3
x: 1
y: 10
0.10000000149011611938476562500000000000000000000000
```

tipos, operadores e precisão IV

Ver também
https://en.wikipedia.org/wiki/Year_2000_problem

tipos, operadores e precisão V

Outro exemplo

src/cents0.c

```c
#include <stdio.h>

int main(void)
{
    float amount;
    printf("Dollar amount: "); scanf("%f", &amount);

    int cents = amount * 100;
    printf("Cents: %i\n", cents);
}
```

tipos, operadores e precisão VI

Executando

```
> make cents0
> ./cents0
Dollar Amount: .99
Cents: 99
> ./cents0
Dollar Amount: 1.23
Cents: 123
> ./cents0
Dollar Amount: 4.20
Cents: 419
```

Parece que o valor 4.20 foi armazenado como 4.199999.

tipos, operadores e precisão VII

A função `round`. Vamos tentar resolver com explicito arredondamento.

src/cents1.c

```
1  #include <math.h>
2  #include <stdio.h>
3
4  int main(void)
5  {
6      float amount;
7      printf("Dollar amount: ");
8      scanf("%f", &amount);
9
10     int cents = round(amount * 100);
11     printf("Cents: %i\n", cents);
12 }
```